Proof Point AI

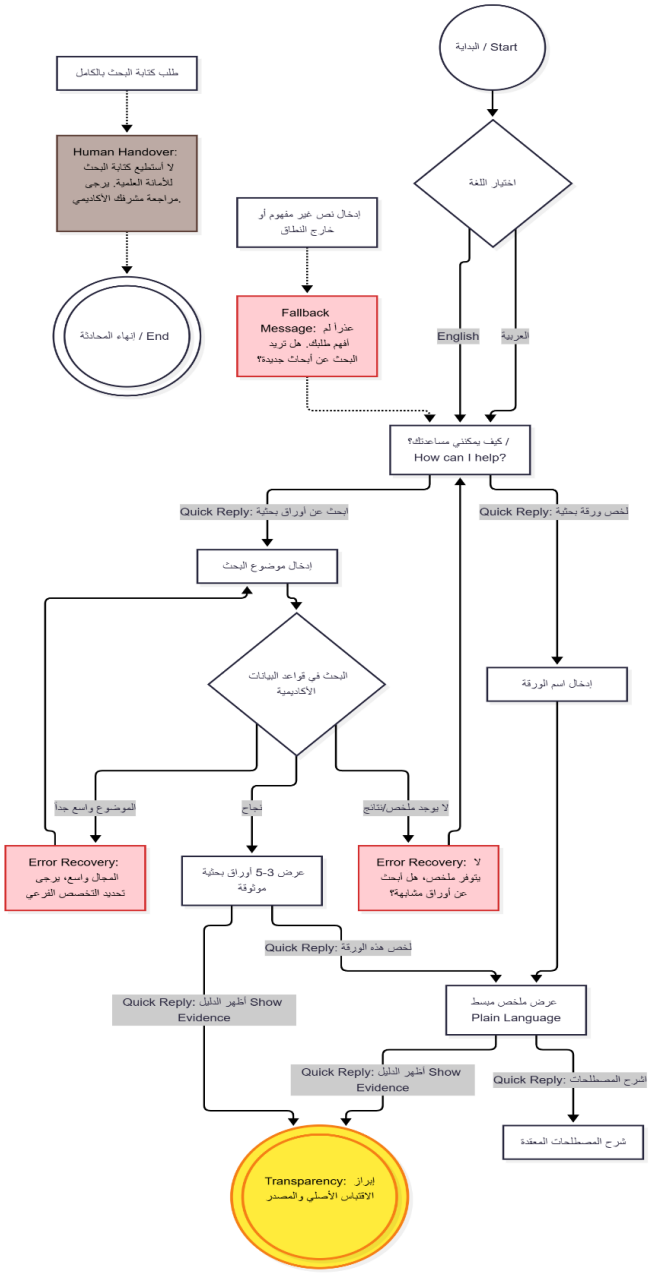
Week 5: Flow and architecture

Deliverables overview

#	Deliverable	Requirements
1	Conversation flow diagram	Must show: the greeting, the main paths, quick replies, error recovery, the fallback message, and human handover.
2	System architecture diagram	Must show: the interface, the backend, the model, the knowledge source or tools, and the safeguards. You must also mark where your principle feature lives in the system.
3	Shneiderman's 8 Golden Rules checklist	For each rule, write which screen or flow decision meets it (consistency, shortcuts, feedback, closure, prevent errors, reversal, control, reduce memory load).
4	First model response	Make your first model call from your own backend and show it live or as a recording. A reply typed into a public AI website does not count.
5	Prompt log entries	Document all prompt log entries used by the team for the week (Appendix A).

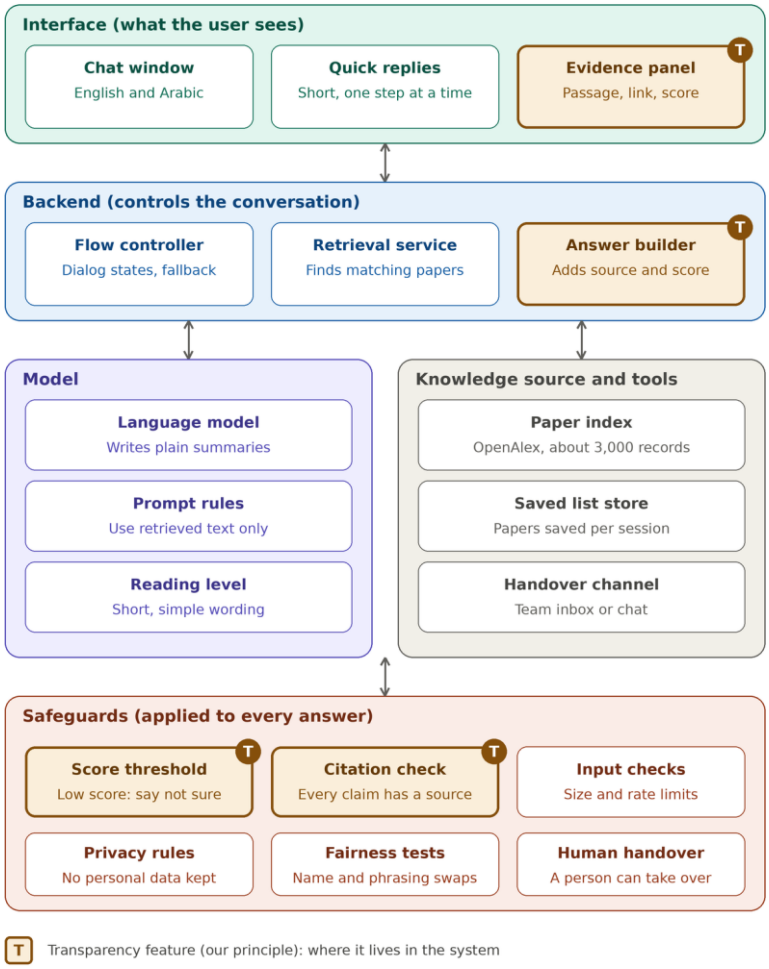
1. Conversation flow diagram

The diagram covers the greeting, the main paths, quick replies, error recovery, the fallback message, and human handover.



2. System architecture diagram

The diagram shows the five required parts: the interface, the backend, the model, the knowledge source and tools, and the safeguards. Boxes marked with a T and an amber outline are where our principle, Transparency, lives in the system.



What each part does

Part	Role	Components
Interface	What the user sees and uses.	Chat window (English and Arabic), quick replies, evidence panel. Full keyboard use, announced new messages, and zoom-friendly layout.
Backend	Runs the conversation and gets the evidence.	Flow controller (dialog states, fallback, handover), retrieval service, answer builder.
Model	Writes the plain-language answer.	Language model, prompt rules (use retrieved text only), reading level (short, simple wording).
Knowledge source and tools	The evidence the answer must come from.	Paper index built from OpenAlex (about 3,000 records, five topics), saved list store, handover channel.
Safeguards	Keep answers honest and safe.	Score threshold, citation check, input checks, privacy rules, fairness tests, human handover.

How one answer is produced

1. The user asks a question or taps a quick reply in the chat window.
2. The flow controller decides what to do next and calls the retrieval service.
3. The retrieval service searches the paper index and returns the best passages with a retrieval score.
4. The language model writes a short, plain-language summary using only those passages.

5. The safeguards run: if the score is below the threshold, the bot says "I am not sure"; the citation check confirms each claim has a source passage.
6. The answer builder attaches the source link and the score to the answer.
7. The interface shows the answer card. "Show evidence" opens the evidence panel with the exact passage, link, and score.

Where the Transparency feature lives

Part	Transparency feature	What the user gets
Interface	Evidence panel (marked T)	The exact passage, the paper link, and the retrieval score for each claim. The user can check the source without leaving the chat.
Interface	AI disclosure in the greeting	The user is told at the start that this is an AI that can make mistakes, and that every answer links to its source.
Backend	Answer builder (marked T)	Attaches the source passage, link, and score to every answer, so nothing is shown without its evidence.
Safeguards	Score threshold (marked T)	When evidence is weak, the bot says "I am not sure" instead of guessing.
Safeguards	Citation check (marked T)	Blocks any claim that has no supporting passage in the paper index.

> **Limit stated to users:** the index holds abstracts and metadata, not full text, so an answer reflects the abstract only. The evidence panel says this when it applies.

3. Shneiderman's 8 Golden Rules checklist

This checklist reviews the conversation flow (Deliverable 1) and the system architecture (Deliverable 2). For each rule, it names the screen or flow decision that meets it. Screens are named as they appear in the flow: Greeting and language choice, the "How can I help?" hub, the results list, the answer card, the Evidence panel, Error Recovery, the Fallback message, and Human Handover.

#	Rule	Screen or flow decision that meets it	Status
1	**Consistency**	Every path returns to the same "How can I help?" hub, in English and in Arabic. The two Error Recovery boxes and the Fallback message follow one pattern: say what happened, then offer a next step. They also share one red style. Every answer is built the same way, with its source and score attached by the Answer builder.	**Met**
2	**Shortcuts**	Quick replies give one-tap actions: Search for papers, Summarize a paper, Summarize this paper, Show evidence, Explain terms. A user who already knows a paper name skips the search and goes straight to the summary. The interface supports full keyboard use.	**Met**
3	**Informative feedback**	The answer card shows the summary, source link, and score. When the score is below the threshold, the bot says "I am not sure". The greeting tells the user this is an AI that can make mistakes. The Evidence panel states the abstract-only limit when it applies. New messages are announced for screen readers.	**Partly met** No visible "searching" indicator during the database search.
4	**Closure**	The dialog has a clear start (Greeting, language choice), middle (search or summary path), and end (answer card, then Show evidence). Human Handover ends with an explicit message and an End node.	**Partly met** The Transparency node and Explain terms have no next step, such as "Anything else?".
5	**Prevent errors**	Quick replies reduce free typing. Language is chosen first. Input checks limit size and rate. The score threshold and citation check stop weak or unsupported answers. A topic that is too broad leads to a request for a narrower sub-field. No results leads to an offer to search similar papers. Unclear input goes to the Fallback. A request to write a whole paper goes to Human Handover.	**Met**
6	**Reversal**	Both Error Recovery paths loop back (to topic entry or to the hub). The Fallback returns to the hub. The Evidence panel opens inside the chat, so closing it returns the user to the conversation.	**Partly met** No Back, Start over, or Change language control. Human Handover ends the chat with no way back to the hub.
7	**User control**	The user chooses the language and the path. Evidence is opt-in (Show evidence). Term explanations are on request. The bot never acts on its own. The layout is zoom-friendly and supports full keyboard use.	**Met**
8	**Reduce memory load**	Quick replies are shown on screen, one step at a time, so users do not memorize commands. Results are limited to 3–5 papers. The answer card puts summary, link, and score in one place. The Evidence panel avoids switching windows. Wording is short and simple. The saved list store keeps papers saved during the session.	**Met**

Recommendations

Four changes would close the gaps found in the three partly met rules.

Priority	Change	Rule	Where in the flow
1	Add Back, Start over, and Change language quick replies.	6	Hub and results list
2	Add "Anything else?" with quick replies that return to the hub.	4	After Show evidence and Explain terms
3	Show a "Searching papers…" indicator.	3	Database search step

Priority	Change	Rule	Where in the flow
4	Offer a way back to the hub after Human Handover.	6	Handover end node

Open review note: the flow sends the user to the academic supervisor at Human Handover, while the architecture lists the handover channel as a team inbox or chat. The team should confirm which one is intended.

Link to our principle: rules 3, 5, and 7 directly support Transparency. The evidence is visible (3), unsupported answers are blocked (5), and the user decides when to see the proof (7).

4. First Model Response

Overview

The first AI model integration was implemented in ResearchHub using the Gemini API through an ASP.NET Core Web API (.NET 10).
The objective was to establish an end-to-end workflow in which the backend receives the user's question, retrieves relevant academic research, sends the collected context to the AI model, and returns a structured answer with supporting evidence.

End-to-End Workflow

The integration follows these main steps:
1. **User Request:** The user submits a question and selects the preferred response language through the POST /api/Chat/ask endpoint.
2. **Research Retrieval:** The ChatController coordinates the process with OpenAlexService, which retrieves relevant academic papers and their available abstracts and identifiers.
3. **Prompt Construction:** AiService combines the user's question with the retrieved research information and prepares a prompt that defines the expected answer format and language.
4. **Model Invocation:** AiService sends an HTTP request to the Gemini API using the configured model and API credentials.
5. **Response Processing:** Gemini generates an answer and supporting claims. The backend processes the response and maps referenced paper identifiers to the corresponding OpenAlex research data.
6. **Final Response:** The backend returns a structured JSON response containing the answer, claims, and their supporting evidence.

Technologies and Responsibilities

Technology	Responsibility
ASP.NET Core Web API	Handles incoming requests and coordinates the overall workflow.
ChatController	Manages the chat endpoint.
OpenAlexService	Retrieves relevant academic publications.
AiService	Constructs prompts and communicates with Gemini.
Gemini API	Generates answers and supporting claims.
OpenAlex API	Supplies the underlying academic research data.
HttpClient	Handles external HTTP communication.

Sample API Request

Endpoint: POST /api/Chat/ask

```
{
  "userMessage": "What is machine learning?",
  "language": "English"
}
```

Response Structure

The backend returns a JSON response containing:

Field	Description
Answer	The main AI-generated response.
Claims	Supporting statements generated by the model.

Field	Description
Evidence	Research passages, academic paper links, and retrieval scores associated with each claim.

This structure connects the generated answer to the academic sources retrieved by the system.

Error Handling

Note: The backend includes error handling for external API failures. Temporary Gemini errors, such as HTTP 429 and 503, are handled using a retry mechanism with increasing delays. Unexpected exceptions are logged and handled centrally to support troubleshooting. API credentials are maintained in backend configuration rather than exposed to the client.

Demonstration and Evidence

The implementation is demonstrated through the ResearchHub Swagger interface, showing:

1. The submitted request to POST /api/Chat/ask.
2. The Gemini API integration inside AiService.
3. The academic retrieval process through OpenAlexService.
4. A successful response generated through the ResearchHub backend.

Outcome

The first model integration established the foundation of ResearchHub's AI-powered research assistant. The backend can now coordinate research retrieval, AI generation, and structured response processing through a single API workflow.

Link For Recording :

https://drive.google.com/file/d/1IExjgS2Qx6V9wj0JbZt_EIxtJLP5GRWv/view?usp=sharing

5. Prompt log (Appendix A)

#	Tool	Exact prompt	What it gave
1	Gemini	Convert our text-based conversation flow for the ProofPoint AI academic chatbot into a Mermaid.js flowchart code. Include the greeting (Arabic/English), main paths (search/summarize), quick replies, error recovery, fallback message, and human handover. Highlight the 'Transparency' principle (Show Evidence node) with a distinct color.	graph TD %% 1. The Greeting Start((البداية / Start)) --> Lang{اختيار اللغة} Lang -->|English| Menu[كيف يمكنني مساعدتك؟ / How can I help?] Lang -->|العربية| Menu %% 2. The Main Paths & 3. Quick Replies Menu -->|Quick Reply: ابحث عن أوراق بحثية| SearchPath[إدخال موضوع البحث] Menu -->|Quick Reply: لخص ورقة بحثية| ReviewPath[إدخال اسم الورقة] SearchPath --> API{البحث في قواعد البيانات الأكاديمية} API -->|نجاح| ShowResults[عرض ٣-٥ أوراق بحثية موثوقة] %% 4. Error Recovery API -->|الموضوع واسع جداً| Error1[Error Recovery: المجال واسع، يرجى تحديد التخصص الفرعي] Error1 --> SearchPath API -->|لا يوجد ملخص/نتائج| Error2[Error Recovery: لا يتوفر ملخص، هل أبحث عن أوراق مشابهة؟] Error2 --> Menu ShowResults -->|Quick Reply: لخص هذه الورقة| Summarize[عرض ملخص مبسط Plain Language] ReviewPath --> Summarize Summarize -->|Quick Reply: اشرح المصطلحات| Explain[شرح المصطلحات المعقدة] %% Principle: Transparency ShowResults -->|Quick Reply: أظهر الدليل| Evidence(((Transparency: إبراز الاقتباس الأصلي والمصدر))) Summarize -->|Quick Reply: أظهر الدليل| Evidence %% 5. The Fallback Message FallbackInput[إدخال نص غير مفهوم أو خارج النطاق] -.-> FallbackMsg[Fallback Message: عذراً لم أفهم طلبك. هل تريد البحث عن أبحاث جديدة؟] FallbackMsg -.-> Menu %% 6. Human Handover HumanInput[طلب كتابة البحث بالكامل] -.-> Handover[Human Handover: لا أستطيع كتابة البحث للأمانة العلمية. يرجى مراجعة مشرفك الأكاديمي.] Handover -.-> End(((إنهاء المحادثة / End))) %% Styling classDef principle fill:#ffeb3b,stroke:#f57f17,stroke-width:3px,color:#000; class Evidence principle; classDef error fill:#ffcdd2,stroke:#d32f2f,stroke-width:2px,color:#000; class Error1,Error2,FallbackMsg error; classDef handover fill:#bcaaa4,stroke:#5d4037,stroke-width:2px,color:#000; class Handover handover;

#	Tool	Exact prompt	What it gave
2	Claude	Create a conversation flow diagram. The diagram should outline the greeting, main paths, quick replies, error recovery, the fallback message, and the process for transitioning to a human agent (Human handover) based on this project (A PDF file containing our project).	I built the flow around your personas and the Proof Point features already in your document (the evidence panel with retrieval score, editable search terms, the saved list and the “I am not sure” message). Two choices are my own, so change them if your design differs. I set the fallback to appear after two failed tries. I also made “Talk to a person” available at every step. **Section 4, flow diagram:** it has a short explanation and the two diagrams as images. Diagram 1 is the main flow (greeting, paths, quick replies). Diagram 2 is the recovery flow (error recovery, fallback, human handover). **Messages table:** it gives the exact wording in English and Arabic for the greeting, the quick replies, the answer card, error recovery, connection lost, the fallback message, human handover, and “no one available”. The Arabic is right-to-left in the table. It uses a neutral greeting and avoids gendered wording for the bot, which fits your bias-check notes.
3	Claude	What is System architecture diagram and How to make it	**Section 2, system architecture diagram** • The diagram shows all five parts: the interface, the backend, the model, the knowledge source and tools, and the safeguards. • Your principle, Transparency, is marked with a **T** badge and an amber outline in four places: the evidence panel, the answer builder, the score threshold and the citation check. • Below the diagram are three short tables and a list: ○ what each part does ○ the seven steps that produce one answer ○ a table of where each Transparency feature lives ○ a note on the abstracts-only limit

#	Tool	Exact prompt	What it gave
3	Gemini	Explain the ResearchHub backend architecture and how its main components communicate.	"Explain the ResearchHub backend architecture and how its main components communicate."
4	Gemini	How can I integrate OpenAlex into an ASP.NET Core Web API to retrieve relevant academic papers	OpenAlex can be integrated using HttpClient to send search requests and retrieve academic publications. The backend processes the returned data and uses relevant paper information as context for AI generation.
5	Gemini	How can I integrate Gemini into my ASP.NET Core backend and use it to generate research-based answers	The backend sends the user's question and retrieved academic context to Gemini through an HTTP POST request. The model generates a structured response that is processed by the backend.
6	Gemini	Design a JSON response containing an AI-generated answer, supporting claims, source passages, academic links, and retrieval scores	A structured JSON response separates the main answer from its supporting claims. Each claim can contain evidence linking it to relevant academic sources, making the output easier to process and display.
7	Gemini	How can I handle Gemini API failures and troubleshoot deployment issues when my ASP.NET Core API works locally but fails in production?	The backend should implement exception handling, logging, and retry mechanisms for temporary external API failures. Production deployment also requires correctly configured environment variables, API credentials, and external service connectivity.